



LAB Worksheet 1

Implementation of Intelligent Agents

THE VACUUM WORLD

- In this worksheet, we will be discussing the **structure of agents** through an example of the vacuum agent.
- The job of AI is to design an agent program that implements the agent function: **the mapping from percepts to actions**.
- We assume this program will run on some sort of computing device with physical sensors and actuators: we call this the architecture: **agent = architecture + program**

Agent Programs

An agent program takes the current percept as input from the sensors and returns an action to the actuators. There is a difference between an agent program and an agent function: an agent program takes the current percept as input whereas an agent function takes the entire percept history.

The agent program takes just the current percept as input because nothing more is available from the environment; if the agent's actions depend on the entire percept sequence, the agent will have to remember the percept.

We'll discuss the following agent programs here with the help of the vacuum world example:

- Random Agent Program
- Table-Driven Agent Program
- Simple Reflex Agent Program
- Model-Based Reflex Agent Program

1. Random Agent Program

A random agent program, as the name suggests, chooses an action at random, without taking into account the percepts. Here, we will demonstrate a random vacuum agent for a trivial vacuum environment, that is, the two-state environment. Let's begin by importing all the functions from the agents module:

```
from agents import *
from notebook import psource
```

Let us first see how we define the *TrivialVacuumEnvironment*. Run the next cell to see how the abstract class *TrivialVacuumEnvironment* is defined in agents module: **psource(TrivialVacuumEnvironment)**

```
# These are the two locations for the two-state environment
loc_A, loc_B = (0, 0), (1, 0)

# Initialize the two-state environment
trivial_vacuum_env = TrivialVacuumEnvironment()

# Check the initial state of the environment
print("State of the Environment: {}".format(trivial_vacuum_env.status))
```

State of the Environment: {(0, 0): 'Clean', (1, 0): 'Dirty'}.

Let's create our agent now. This agent will choose any of the actions from 'Right', 'Left', 'Suck' and 'NoOp' (No Operation) randomly.

```
# Create the random agent
random_agent = Agent(program=RandomAgentProgram(['Right', 'Left', 'Suck', 'NoOp']))
```

We will now add our agent to the environment.

```
# Add agent to the environment
trivial_vacuum_env.add_things(random_agent)
print("RandomVacuumAgent is located at {}".format(random_agent.location))
```

RandomVacuumAgent is located at (1, 0).



Let's run our environment now.

```
# Running the environment
trivial_vacuum_env.step()
# Check the current state of the environment
print("State of the Environment: {}".format(trivial_vacuum_env.status))
print("RandomVacuumAgent is located at {}".format(random_agent.location))
```

State of the Environment: {(0, 0): 'Clean', (1, 0): 'Dirty'}.
RandomVacuumAgent is located at (1, 0).

2. Table-Driven Agent Program

A table-driven agent program keeps track of the percept sequence and then uses it to index into a table of actions to decide what to do. The table represents explicitly the agent function that the agent program embodies. In the two-state vacuum world, the table would consist of all the possible states of the agent.

```
table = {((loc_A, 'Clean'),): 'Right',
          ((loc_A, 'Dirty'),): 'Suck',
          ((loc_B, 'Clean'),): 'Left',
          ((loc_B, 'Dirty'),): 'Suck',
          ((loc_A, 'Dirty'), (loc_A, 'Clean')): 'Right',
          ((loc_A, 'Clean'), (loc_B, 'Dirty')): 'Suck',
          ((loc_B, 'Clean'), (loc_A, 'Dirty')): 'Suck',
          ((loc_B, 'Dirty'), (loc_B, 'Clean')): 'Left',
          ((loc_A, 'Dirty'), (loc_A, 'Clean'), (loc_B, 'Dirty')): 'Suck',
          ((loc_B, 'Dirty'), (loc_B, 'Clean'), (loc_A, 'Dirty')): 'Suck'
}
```

We will now create a table-driven agent program for our two-state environment.

```
# Create a table-driven agent
table_driven_agent = Agent(program=TableDrivenAgentProgram(table=table))
```

Since we are using the same environment, let's remove the previously added random agent from the environment to avoid confusion.

```
trivial_vacuum_env.delete_thing(random_agent)

# Add the table-driven agent to the environment

trivial_vacuum_env.add_thing(table_driven_agent)

print("TableDrivenVacuumAgent is located at {}".format(table_driven_agent.location))
```

TableDrivenVacuumAgent is located at (0, 0).

```
# Run the environment
trivial_vacuum_env.step()
# Check the current state of the environment
print("State of the Environment: {}".format(trivial_vacuum_env.status))
print("TableDrivenVacuumAgent is located at {}".format(table_driven_agent.location))
```

State of the Environment: {(0, 0): 'Clean', (1, 0): 'Dirty'}.
TableDrivenVacuumAgent is located at (1, 0).

3. Simple Reflex Agent Program

A simple reflex agent program selects actions on the basis of the current percept, ignoring the rest of the percept history. These agents work on a condition-action rule (also called situation-action rule, production or if-then rule), which tells the agent the action to trigger when a particular situation is encountered.

```
# Delete the previously added table-driven agent
trivial_vacuum_env.delete_thing(table_driven_agent)
```



To create our agent, we need two functions: INTERPRET-INPUT function, which generates an abstracted description of the current state from the percept and the RULE-MATCH function, which returns the first rule in the set of rules that matches the given state description.

```
loc_A = (0, 0)
loc_B = (1, 0)

"""We change the simpleReflexAgentProgram so that it doesn't make use of the Rule class"""
def SimpleReflexAgentProgram():
    """This agent takes action based solely on the percept. [Figure 2.10]"""

    def program(percept):
        loc, status = percept
        return ('Suck' if status == 'Dirty'
                else 'Right' if loc == loc_A
                else 'Left')

        # Create a simple reflex agent the two-state environment

    program = SimpleReflexAgentProgram()
    simple_reflex_agent = Agent(program)
```

Now add the agent to the environment:

```
trivial_vacuum_env.add_thing(simple_reflex_agent)
print("SimpleReflexVacuumAgent is located at {}".format(simple_reflex_agent.location))
```

SimpleReflexVacuumAgent is located at (1, 0).

```
# Run the environment
trivial_vacuum_env.step()
# Check the current state of the environment
print("State of the Environment: {}".format(trivial_vacuum_env.status))
print("SimpleReflexVacuumAgent is located at {}".format(simple_reflex_agent.location))
```

State of the Environment: {(0, 0): 'Clean', (1, 0): 'Clean'}.
SimpleReflexVacuumAgent is located at (1, 0).

4. Model-Based Reflex Agent Program

A model-based reflex agent maintains some sort of internal state that depends on the percept history and thereby reflects at least some of the unobserved aspects of the current state. In addition to this, it also requires a model of the world, that is, knowledge about "how the world works".

```
# Delete the previously added simple reflex agent
trivial_vacuum_env.delete_thing(simple_reflex_agent)
```

We need another function UPDATE-STATE which will be responsible for creating a new state description.

```
# TODO: Implement this function for the two-dimensional environment
def update_state(state, action, percept, model):
    pass

# Create a model-based reflex agent
model_based_reflex_agent = ModelBasedVacuumAgent()

# Add the agent to the environment
trivial_vacuum_env.add_thing(model_based_reflex_agent)
print("ModelBasedVacuumAgent is located at {}".format(model_based_reflex_agent.location))
```

ModelBasedVacuumAgent is located at (0, 0).

```
#Run the environment
trivial_vacuum_env.step()
# Check the current state of the environment
print("State of the Environment: {}".format(trivial_vacuum_env.status))
print("ModelBasedVacuumAgent is located at {}".format(model_based_reflex_agent.location))
```

State of the Environment: {(0, 0): 'Clean', (1, 0): 'Clean'}.
ModelBasedVacuumAgent is located at (1, 0).